

Soc 프로젝트 최종 보고서



과목	시스템온칩설계
담당교수	김현진
조원	32210776 김수민
	32221975 박찬혁
	32223899 장서영
제출일	24.12.23.

프로젝트 개요서

프로젝트명	주차장 시스템 구현
팀원/역할	김수민(32210776) / PWM 모듈 설계 및 서보모터 기능 구현 박찬혁(3221975) / 2. 장서영(32223899) / AHB 버스 인터커넥팅, 추상화 및 HW-SW 인터페이스 정의
프로젝트 소개	Basys3 FPGA 보드와 Cortex-M0 프로세서를 사용해 차량 접근 시 차단기를 동작시키고 남은 주차 공간을 표시하는 주차장 시스템을 구현한다. FPGA와 서보 모터, 센서, 7세그먼트 디스플레이를 연동해 하드웨어와 소프트웨어를 통합하였다.
동작 방법	차단기 근처 조도 센서에서 차량을 감지하면 차단기가 열리고 차량이 감지되지 않을 경우 차단기는 닫혀 있는 상태를 유지한다. 주차 공간에서는 차량이 주차되지 않은 경우 LED가 점등되며 차량이 주차되면 LED가 꺼진다. 남은 주차 공간의 개수는 7-Segment 디스플레이를 통해 실시간으로 표시된다.
하드웨어 구성도	The diagram illustrates the hardware architecture. On the left, two green-outlined boxes represent the '주차공간 인식' (Parking Space Recognition) modules. Each box contains a 'Pmod ALS' sensor module and a red LED. These are connected via yellow lines to a central 'Basys3 boarda' (FPGA board). The FPGA board is then connected via yellow lines to a blue-outlined box on the right representing the '차단기 제어' (Barrier Control) module. This module contains a 'Pmod ALS' sensor module and a 'Servo motor sg-90'.

목차

I. 개요

II. 프로젝트 내용

- 1) 기능 및 시나리오
- 2) 사용 주변 장치
- 3) 개발 환경

III. 설계

- 1) HW Level
- 2) SW Level

IV. 결과물

I. 개요

본 프로젝트는 본 프로젝트는 Xilinx Artix-7이 탑재된 Digilent 사의 FPGA 개발보드 Basys3에 Cortex M0 프로세서를 올려 차량이 차단기 앞으로 오면 차단기가 동작하고, 남은 주차 자리를 표시하는 기본적인 주차장 시스템을 구현한다.

설계자 별 담당 부분 및 상세 개발 내용은 다음과 같음.

설계자	담당파트
김수민	PWM 모듈 설계 및 서보모터 제어 기능 구현
박찬혁	
장서영	AHB 버스 인터커넥팅, 추상화 및 HW-SW 인터페이스 정의

II. 프로젝트 내용

1) 기능 및 시나리오

- ① 차량이 차단기 앞으로 오면 차단기를 연다.
- ② 각 주차 자리에는 조명이 달려있어 운전자가 주차할 수 있는 자리를 쉽게 찾을 수 있다.
- ③ 차량이 주차하면 해당 주차 자리의 조명을 끈다.
- ④ 남아 있는 주차 자리 개수를 주차장 입구에 표시한다.

2) 사용 주변 장치

장치	설명
조도센서	차단기 부근 차량의 접근과 주차 유/무 감지
7 Segment	주차장 입구에 위치, 남아 있는 주차 자리 개수 표시
LED	운전자에게 주차할 수 있는 주차석 표시
Servo Motor	주차장 출입구 차단기 역할

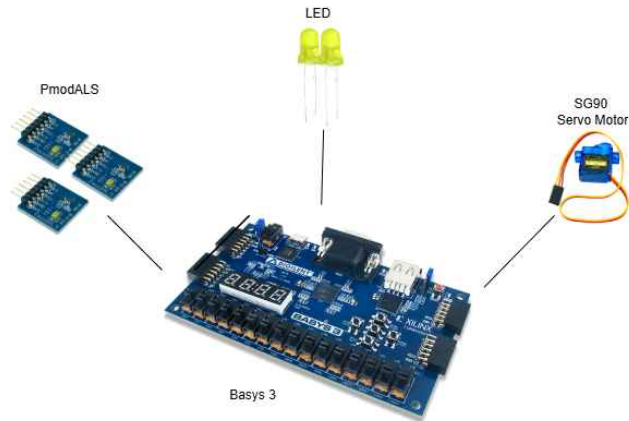
3) 개발 환경

구분		상세내용
S/W 개발환경	개발도구	Arm Keil MDK
	개발언어	C, Assembly
H/W 개발환경	FPGA	Basys 3 (based on Artix 7)
	Processor	Cortex M0
	Bus Protocol	AHB
	개발도구	Vivado
	개발언어	Verilog

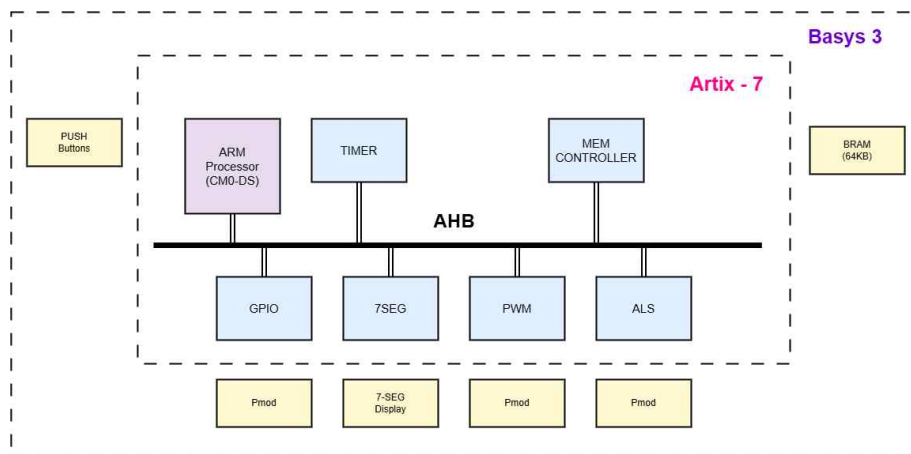
Ⅲ. 설계 및 코드

1) HW Level

① 외부 I/O



② Top Level



③ Memory Map

Yellow : 실제 시나리오 구동에 필요한 영역

Purple : 디버깅용 영역

	START ADDRESS	END ADDRESS	SIZE	INTERRUPT CAPABILITY
MEM	0x0000_0000	0x00FF_FFFF	64KB	NO
LED	0x5000_0000	0x50FF_FFFF	16MB	NO
KEY	0x5100_0000	0x51FF_FFFF	16MB	YES
ALS0	0x5200_0000	0x52FF_FFFF	16MB	NO
ALS1	0x5300_0000	0x53FF_FFFF	16MB	NO
ALS2	0x5400_0000	0x54FF_FFFF	16MB	NO
7SEG	0x5500_0000	0x55FF_FFFF	16MB	NO
GPIO	0x5600_0000	56FFF_FFFF	16MB	NO
TIMER	0x5700_0000	0x57FF_FFFF	16MB	YES
PWM	0x5800_0000	0x58FF_FFFF	16MB	NO

④ IP 개선 및 수정사항

(AHBGPIO)

ARM 사에서 제공해주는 GPIO IP의 경우 주소와 데이터 입력을 바이트 단위로 제어를 해 SW단에서 GPIO핀 단독 제어를 못한다는 큰 결점이 있었다. 따라서 GPIO 핀들을 개별적으로 제어가 가능하게끔 다음과 같이 개선하였다.

```
module AHBGPIO (
    input wire HCLK,
    input wire HRESETn,
    input wire [31 :0 ] HADDR,
    input wire [1 :0 ] HTRANS,
    input wire [31 :0 ] HWDATA,
    input wire HWRITE,
    input wire HSEL,
    input wire HREADY,
    input wire [15 :0 ] GPIOIN,

    //Output
    output wire HREADYOUT,
    output wire [31 :0 ] HRDATA,
    output reg [15 :0 ] GPIOOUT

);

localparam [7 :0 ] gpio_set_addr =8'h00 ; // 핀 개별 SET 제어 주소
localparam [7 :0 ] gpio_clear_addr =8'h04 ; // 핀 개별 CLEAR 제어 주소
localparam [7 :0 ] gpio_set_all =8'h08 ; // 핀 모두 SET 주소
localparam [7 :0 ] gpio_clear_all =8'h0c ; // 핀 모두 CLEAR 주소

reg [15 :0 ] gpio_select;
reg [31 :0 ] last_HADDR;
reg [1 :0 ] last_HTRANS;
reg last_HWRITE;
reg last_HSEL;

assign HREADYOUT =1'b1 ;

// Set Registers from address phase
always @(posedge HCLK)
begin
    if (HREADY)
    begin
        last_HADDR <= HADDR;
        last_HTRANS <= HTRANS;
        last_HWRITE <= HWRITE;
        last_HSEL <= HSEL;
    end
end
```

```

end
end

//Combinational logic for assigning gpio_data

always @(posedge HCLK, negedge HRESETn)
begin
    if (!HRESETn)
        GPIOOUT[15 :0 ] =16'h0000 ;
    else if ((last_HADDR[7 :0 ] == gpio_set_addr) & last_HSEL &
last_HWRITE & last_HTRANS[1 ]) // 비트 제어, SET
        begin
            case (HWDATA[15 :0 ])
                16'h0001 : gpio_select =0 ;
                16'h0002 : gpio_select =1 ;
                16'h0004 : gpio_select =2 ;
                16'h0008 : gpio_select =3 ;

                16'h0010 : gpio_select =4 ;
                16'h0020 : gpio_select =5 ;
                16'h0040 : gpio_select =6 ;
                16'h0080 : gpio_select =7 ;

                16'h0100 : gpio_select =8 ;
                16'h0200 : gpio_select =9 ;
                16'h0400 : gpio_select =10 ;
                16'h0800 : gpio_select =11 ;

                16'h1000 : gpio_select =12 ;
                16'h2000 : gpio_select =13 ;
                16'h4000 : gpio_select =14 ;
                16'h8000 : gpio_select =15 ;
            endcase

            GPIOOUT[gpio_select] =1 ;
        end
    else if ((last_HADDR[7 :0 ] == gpio_clear_addr) & last_HSEL &
last_HWRITE & last_HTRANS[1 ]) // 비트 제어, CLEAR
        begin
            case (HWDATA[15 :0 ])
                16'h0001 : gpio_select =0 ;
                16'h0002 : gpio_select =1 ;
                16'h0004 : gpio_select =2 ;
                16'h0008 : gpio_select =3 ;

                16'h0010 : gpio_select =4 ;
                16'h0020 : gpio_select =5 ;
                16'h0040 : gpio_select =6 ;
                16'h0080 : gpio_select =7 ;

                16'h0100 : gpio_select =8 ;

```

```

        16'h0200 : gpio_select =9 ;
        16'h0400 : gpio_select =10 ;
        16'h0800 : gpio_select =11 ;

        16'h1000 : gpio_select =12 ;
        16'h2000 : gpio_select =13 ;
        16'h4000 : gpio_select =14 ;
        16'h8000 : gpio_select =15 ;
    endcase
    GPIOOUT[gpio_select] =0 ;
end

    else if ((last_HADDR[7 :0 ] == gpio_set_all) & last_HSEL &
last_HWRITE & last_HTRANS[1 ]) // 전체 제어, SET
    begin

        GPIOOUT[15 :0 ] =16'hFFFF ;
    end

    else if ((last_HADDR[7 :0 ] == gpio_clear_all) & last_HSEL &
last_HWRITE & last_HTRANS[1 ]) // 비트 제어, CLEAR
    begin

        GPIOOUT[15 :0 ] =16'h0000 ;
    end

end

assign HRDATA[15 :0 ] = GPIOOUT;
endmodule

```

(AHBPWM)

PWM 관련 AHB IP는 찾기가 어려워 직접 설계하였다. AHB2PWM 모듈은 프로세서와 직접 연결되어 HWDATA = 1과 0의 값이 들어오면 VALID 신호를 만들어 PWM 신호를 Pmod 포트에 직접적으로 발생시키는 myPWM 모듈과 와이어링하였다.

```
module AHB2PWM (
    input wire HCLK,
    input wire HRESETn,
    input wire [31:0] HADDR,
    input wire [1:0] HTRANS,
    input wire [31:0] HWDATA,
    input wire HWRITE,
    input wire HSEL,
    input wire HREADY,
    output wire HREADYOUT,
    output wire [31:0] HRDATA,
    output reg VALID
);

localparam [7:0] base_addr = 8'h00 ;

reg [31:0] last_HADDR;
reg [1:0] last_HTRANS;
reg last_HWRITE;
reg last_HSEL;

reg [20:0] counter;
reg [16:0] control = 0 ;

assign HREADYOUT = 1'b1 ;

// Set Registers from address phase
always @(posedge HCLK)
begin
    if (HREADY)
    begin
        last_HADDR <= HADDR;
        last_HTRANS <= HTRANS;
        last_HWRITE <= HWRITE;
        last_HSEL <= HSEL;
    end
end

always @(posedge HCLK, negedge HRESETn)
begin
    if (!HRESETn)
        VALID = 16'h0000 ;
    else if ((last_HADDR[7:0] == base_addr) & last_HSEL & last_HWRITE &
last_HTRANS[1]) // 비트 제어, SET
    begin
        if (HWDATA[15:0] == 16'h0000 ) VALID = 1 ;
    end
end
```

```

        else if (HWDATA[15 :0 ] ==16'h0001 ) VALID =0 ;
    end
end

    assign HRDATA[15 :0 ] = VALID;
endmodule

```

(myPWM)

PWM 신호 생성 모듈은 입력 클럭(clk)과 유효 신호(VALID)를 기반으로 서보 모터 제어 신호(MOTOR)를 생성한다. 21비트 카운터를 사용해 50MHz 클럭에서 1초 주기를 설정하고 듀티 사이클은 VALID 신호에 따라 증가 또는 감소하도록 설계되었다. 이를 통해 차량 접근 시 차 단기를 제어하는 PWM 신호를 출력한다.

```

module myPWM (
    input clk,
    input VALID,          // Clock signal, pwm duty cycle 변경함.
    output reg MOTOR      // PMOD control output
);
    // Setting Counter
    reg [20 :0 ] counter;    // Counter for timing
    reg [16 :0 ] control =0 ; // Control for PWM adjustment
    reg toggle =1 ;         // Toggle for servo direction

    always @(posedge clk) begin
        counter <= counter +1 ;    // Increment counter
        if (counter == 'd999999 )    // 50Mhz 기준 1sec
            counter <=0 ;           // Reset counter

        if (counter < ('d100000 + control)) begin
            MOTOR <=4'b0001 ;        // Activate servo motor
        end else begin
            MOTOR <=0 ;              // Reset servo PWM signal
        end

        if (counter ==0 ) begin
            if (VALID ==1'b1 )        // Check VALID signal
                control <= control +500 ; // Increase control value for PWM
            else                        // If down button pressed
                control <= control -500 ; // Decrease control value for PWM
        end
    end
endmodule

```

2) SW Level

① HW 추상화 및 SW 인터페이스 정의

SW 개발자들의 효율적인 개발을 위하여 레지스터를 제어하는 부분을 다음과 같이 C언어 헤더 파일로 작성해 HW-SW 인터페이스를 만들었다.

(기본 Memory Map 정의)

```
1  /*****  
2  /*                      Peripheral memory map                      */  
3  /***/  
4  #define AHB_LED_BASE      0x50000000  
5  #define AHB_KEY_BASE      0x51000000  
6  #define AHB_ALS0_BASE     0x52000000  
7  #define AHB_ALS1_BASE     0x53000000  
8  #define AHB_ALS2_BASE     0x54000000  
9  #define AHB_SEG_BASE      0x55000000  
10 #define AHB_GPIO_BASE     0x56000000  
11 #define AHB_TIMER_BASE    0x57000000  
12 #define AHB_PWM_BASE      0x58000000  
13  
14  
15  
16 /*****  
17 /*                      NVIC System Registers                      */  
18 /***/  
19 #define NVIC_INT_ENABLE    0xE000E100  
20 #define NVIC_INT_PRIORITY0 0xE000E400  
21  
22  
23  
24 /*****  
25 /*                      Peripheral Register Map Offset              */  
26 /***/  
27  
28  
29 #define GPIO_SET 0x00  
30 #define GPIO_CLEAR 0x04  
31 #define GPIO_SET_ALL 0x08  
32 #define GPIO_CLEAR_ALL 0x0c  
33  
34 #define TIMER_INITVALUE    0x0  
35 #define TIMER_CURVALUE     0x4  
36 #define TIMER_CONTROL      0x8  
37 #define TIMER_CLEAR        0xC
```

(7 Segment 관련)

```
#ifndef SEGMENTS_H
#define SEGMENTS_H
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include "../REGISTERS.h"
typedef unsigned short Uint16 ;
typedef unsigned int Uint32 ;

typedef struct SEGMENT_CTRL_REGS {
    volatile Uint32 SEG3 ;
    volatile Uint32 SEG2 ;
    volatile Uint32 SEG1 ;
    volatile Uint32 SEG0 ;
}SEGMENT_REGS ;
volatile struct SEGMENT_CTRL_REGS *const SegmentCtrlRegsPtr =(volatile struct
SEGMENT_CTRL_REGS *) (AHB_SEG_BASE );
#define SegCtrlRegs (*SegmentCtrlRegsPtr )

#endif

ex) SegCtrlRegs.SEG3 = 4; // 3번 세그먼트에 4를 띄워라.
```

(모터 관련)

```
#ifndef MOTOR_H
#define MOTOR_H
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include "../REGISTERS.h"

volatile unsigned int *MotorCtrlPtr =(volatile unsigned int *) AHB_PWM_BASE ;
#define MotorCtrl (*MotorCtrlPtr )

#endif

// MotorCtrl = 1; Surbo Motor CW, PWM 듀티사이클 증가
// MotorCtrl = 0; Surbo Motor CCW, PWM 듀티사이클 감소
```

(조도 센서 관련)

```
#ifndef ALS_DATA_H
#define ALS_DATA_H
#include <stdint.h>
#include "../REGISTERS.h"

volatile unsigned int *ALS0 =(volatile unsigned int *) AHB_ALS0_BASE ;
volatile unsigned int *ALS1 =(volatile unsigned int *) AHB_ALS1_BASE ;
volatile unsigned int *ALS2 =(volatile unsigned int *) AHB_ALS2_BASE ;
#define AlsData0 (*ALS0 );
#define AlsData1 (*ALS1 );
#define AlsData2 (*ALS2 );

#endif
ex) int a = AlsData0; 변수 a에 0번 조도센서의 값을 저장하라는 뜻.
```

(GPIO 관련)

```
#include <stdio.h >
#include <stdint.h >
#include <stdlib.h >
#include "../REGISTERS.h "

typedef unsigned short Uint16 ;
typedef unsigned int Uint32 ;

/*****
**/
/*          GPIO SET          */
/*****
**/
#define GPIO0 0x0001
#define GPIO1 0x0002
#define GPIO2 0x0004
#define GPIO3 0x0008
#define GPIO4 0x0010
#define GPIO5 0x0020
#define GPIO6 0x0040
typedef struct GPIO_CTRL_REGS {
    volatile Uint32 SET ;
    volatile Uint32 CLEAR ;
    volatile Uint32 ALL_SET ;
    volatile Uint32 ALL_CLEAR ;
}GPIO_REGS ;
volatile struct GPIO_CTRL_REGS *const GpioCtrlRegsPtr =(volatile struct
GPIO_CTRL_REGS *) (AHB_GPIO_BASE );
#define GpioCtrlRegs (*GpioCtrlRegsPtr )
// GpioCtrlRegs.SET = GPIO0; GPIO 0번을 SET
// GpioCtrlRegs.CLEAR = GPIO1; GPIO 1번을 CLEAR
// GpioCtrlRegs.ALL_SET = 1; GPIO 모두 SET
// GpioCtrlRegs.ALL_CLEAR = 1; GPIO 모두 CLEAR
```

② 알고리즘 코드

IV. 결과물

